

TP01 — Installer ShopLab et observer une requête

Séances : S01–S02 · Durée : 180 min · Compétences : C01, C02 · Niveau : guidé

Contrat de sécurité

Travaillez uniquement sur `http://127.0.0.1` et `https://127.0.0.1`. Arrêtez-vous si une commande contient une autre cible. Les comptes `alice`, `bob` et `admin` et toutes les données du lab sont synthétiques. Ne joignez jamais de token complet au compte rendu.

Objectifs et prérequis

À la fin, vous saurez vérifier l'hôte, démarrer et arrêter le lab, suivre DNS/TCP/TLS/HTTP/proxy/API/DB et produire une preuve expurgée. Prérequis : terminal, Git, Docker avec Compose v2, `curl`, OpenSSL et Python 3. Depuis la racine du dépôt, utilisez `./scripts/verify-host.sh` avant de commencer.

Connexion au cours : SL001–SL008 et chapitre S01 préparent la mise en place et la partie A; SL009–SL016 et chapitre S02 préparent les parties B et D. Vous observez une seule requête ShopLab et conservez le même identifiant de corrélation de l'entrée proxy jusqu'à l'API.

Livrables et critères de réussite

- un tableau `étape → observation → preuve → conclusion` ;
- une trace HTTP annotée, un certificat décrit sans clé privée et un schéma du flux ;
- `verify-lab.sh` terminé par `FAIL=0`, puis preuve de cleanup sans conteneur restant.

Mise en place (20 min)

```
cd labs/shoplab
../../scripts/verify-host.sh
./scripts/lab-start.sh core
./scripts/health-check.sh core
```

Résultat attendu : des lignes `[PASS]`, l'URL locale et `mode=vulnerable`. Un `[WARN]` demande une vérification manuelle; un `[FAIL]` bloque la suite.

A — Scope et dossier de preuve (20 min)

```
mkdir -p preuves/TP01
```

Dans `scope.md`, notez cible autorisée, heure de début, outils, stop conditions et règle d'expurgation. Relevez `git rev-parse --short HEAD`, `docker version` et `docker compose version`.

Exemple du **type** de ligne attendu dans l'index, à adapter à votre exécution :

```
2026-09-21T08:42:00Z | ./scripts/verify-host.sh | PASS/WARN/FAIL | interprétation | limite
```

La date est synthétique. Copiez votre sortie réelle, sans token ni donnée personnelle. Si la cible affichée n'est pas locale, si `verify-host` termine par `[FAIL]` ou si le reset est incertain : arrêtez-vous et demandez une validation.

B — Requête HTTP (35 min)

```
curl -sv -H 'X-Correlation-ID: tp01-groupe-XX' \
  http://127.0.0.1:${SHOPLAB_HTTP_PORT:-8080}/api/health \
  -o preuves/TP01/health.json 2>preuves/TP01/http-trace.txt
```

Annotez méthode, chemin, [Host](#), statut, type de contenu et identifiant de corrélation. Expliquez pourquoi un statut HTTP n'est pas la preuve que toute la chaîne est saine.

C — TCP et TLS (35 min)

```
openssl s_client -connect 127.0.0.1:${SHOPLAB_HTTPS_PORT:-8443} \
  -servername localhost </dev/null 2>preuves/TP01/tls.txt
curl -skI https://127.0.0.1:${SHOPLAB_HTTPS_PORT:-8443}/
```

Relevez sujet, émetteur, SAN, dates, version TLS et cipher. Le certificat est auto-signé : `-k` est toléré uniquement ici. Expliquez la différence entre chiffrement du transport et confiance dans l'identité.

D — De proxy à la base (45 min)

```
./scripts/compose.sh --profile core ps
./scripts/compose.sh --profile core logs --tail=30 proxy api db
curl -sS http://127.0.0.1:${SHOPLAB_HTTP_PORT:-8080}/api/products?q=lamp
```

Forme synthétique à rechercher : `proxy ... correlation_id=tp01-groupe-XX`, puis `api ... correlation_id=tp01-groupe-XX`. Les timestamps et détails varient. Une ligne DB sans identifiant HTTP ne doit pas être attribuée à votre requête sans lien supplémentaire.

Reliez chaque service au flux. Retrouvez votre `X-Correlation-ID` dans les logs API. Distinguez ce que vous avez observé de ce que vous inférez. Dessinez les frontières hôte/edge/core/base.

Vérification, reset et cleanup (20 min)

```
./scripts/verify-lab.sh basic
./scripts/reset.sh
./scripts/health-check.sh core
./scripts/cleanup.sh
./scripts/compose.sh --profile core ps --all
```

Le dernier tableau ne doit contenir aucun conteneur actif du projet. Le trap de test nettoie aussi après échec.

Dépannage

- port occupé : définissez `SHOPLAB_HTTP_PORT=18080 SHOPLAB_HTTPS_PORT=18443` ;
- daemon absent : Docker Desktop/Colima doit être démarré ; sous Linux, vérifiez `docker info` ;
- certificat refusé : n'importez pas la CA ; utilisez `-k` pour ce lab seulement ;
- WSL2 : placez le dépôt dans le système de fichiers Linux pour de meilleures performances.

Barème formatif /20

Critère	Insuffisant	Conforme	Maîtrisé	Points
Scope/éthique	cible non bornée	localhost et stop conditions	justification/expurgation	4
Chaîne technique	liste sans lien	flux complet correct	frontières et limites	6
Preuves	absentes/brutes	horodatées et expurgées	reproductibles et indexées	5
Cycle de vie	résidus	start/verify/reset/cleanup	diagnostic autonome	5

Vue ShopLab avant de commencer

S02 · Du DNS à la base

Source éditable · palette pédagogique provisoire



Suivre une requête sans sauter de couche ni confondre protocole et composant.

Question du visuel : où la donnée traverse-t-elle une frontière de confiance, quel composant décide et quel état doit être observé après l'action ?

Élément	Lecture attendue
Zone étudiée	DNS → TCP → TLS → proxy → API → PostgreSQL
Direction	aller de la requête, puis retour statut/headers/corps/logs
Contrôle	placé au composant qui possède la décision, refus par défaut
État final	parcours légitime fonctionnel, cas négatif refusé, preuve expurgée

Bloc de commande important — méthode commune

1. **Goal** — observer ou modifier uniquement l'état annoncé pour `curl/openssl/logs`.
2. **Before** — noter mode ShopLab, commit, services et baseline.
3. **Command** — exécuter exactement la commande du bloc concerné sur `127.0.0.1` ou le réseau Docker isolé.
4. **What happens inside the system** — suivre le nœud actif dans le diagramme, puis la décision et la trace générée.
5. **Expected output** — prédire statut, champ ou branche avant d'exécuter; ne pas fabriquer une sortie.
6. **Small diagram or highlighted architecture** — entourer sur le visuel le composant qui change ou révèle son état.
7. **How to verify** — répéter le test négatif et le parcours légitime avec le même contexte documenté.
8. **Evidence to save** — trace corrélée + limite; UTC, commande, sortie minimale, conclusion et limite.
9. **If it fails** — arrêter si la cible sort du lab; sinon vérifier une seule frontière à la fois et consigner l'écart.
10. **Next step** — indexer la preuve, effectuer le debrief demandé, puis `reset/cleanup`.